March 25, 2025

How to Use an API in Python

Whether you're working on a beginner Python project, exploring data science, or building with AI, learning how to use an API in Python lets you pull real-time data into your code.

For example, platforms like Reddit, Twitter, and Facebook offer APIs to help you retrieve specific datasets or perform advanced queries. In this guide, we'll explore how to use an API in Python, starting with retrieving data from the International Space Station (ISS).

What is an API?

An API (**application programming interface**) is a server that lets you request and send data using code. It serves as a bridge between your application and external systems, allowing you to incorporate valuable data and functionality without having to "manually" get the data yourself.

How APIs Work

APIs work by sending requests to a server and receiving responses. For instance, when you visit a webpage, your browser makes a request to the server, which sends back the page content. APIs follow the same principle, letting you programmatically retrieve data, access features, or interact with services.

Popular Tools that Use APIs

Here are some widely used APIs that showcase the power of integrating an API in Python:

- **OpenAI API**: A versatile tool for text generation, summarization, and natural language understanding.
- **Google Cloud AI APIs**: Provides vision, language, and conversational AI services.
- **IBM Watson API**: A suite of tools for tasks like NLP and visual recognition.

Why and When to Use an API in Python

Using an API in Python offers several advantages for AI and data science projects:

- **Real-time data access**: Retrieve up-to-date data on demand, essential for projects that rely on timely information for accurate insights.

- **Pre-processed Insights**: Many APIs supply enriched data, such as sentiment analysis or key entity recognition, reducing the time spent on preprocessing tasks.

When APIs Are Better Than Static Datasets

APIs are particularly effective in the following situations:

- **Rapidly evolving data**: For information that changes frequently, APIs eliminate the hassle of repeatedly downloading and updating static datasets.
- **Targeted data extraction**: Retrieve only the specific segment of a dataset you need, such as recent Reddit comments.
- **Access to specialized computations**: APIs like Spotify's provide unique insights, such as genre classifications, leveraging their computational infrastructure.

How APIs Enhance Python Projects

Working with an API in Python opens up opportunities to create smarter and more dynamic applications. For example:

- **Integrating Pre-Built AI Models**: Use APIs to incorporate advanced natural language processing (NLP) or computer vision models into your projects without developing them from scratch.
- **Streamlining Data Acquisition**: Retrieve data from platforms like Reddit, Facebook, or Kaggle to fuel analysis or machine learning projects.
- **Leveraging AI Services**: APIs provide access to features like sentiment analysis, language translation, or image recognition, enabling you to build robust solutions quickly.

To work with an API in Python, we'll need a tool for making requests. The most common library for this purpose is the **requests** library. Since `requests` isn't included in the standard Python library, we'll need to install it before we can use it.

If you use `pip` to manage your Python packages, you can install the `requests` library with the following command:

```
pip install requests
```

If you're using conda, use this command instead:

```
conda install requests
```

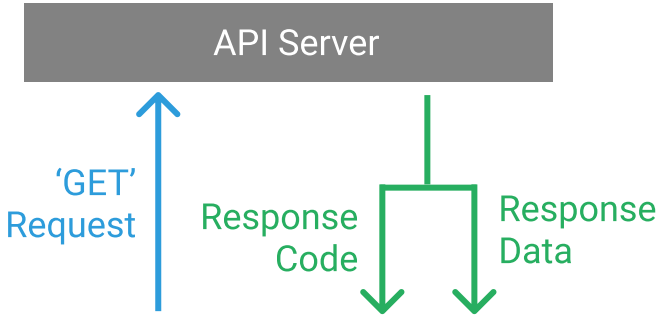
Once installed, import the library into your script to start making API requests:

```
import requests
```

Making Your First API Request in Python

Now that we've set up the `requests` library, let's make our first API request. We'll start with a simple GET request and take a closer look at the status codes that come with the server's response.

now.



A typical API request involves sending a query to the server and receiving a response.

When we make an API request, the response we get back from the server includes a **status code** that tells us if the request was successful or if something went wrong.

Let's try making a GET request to an API endpoint that doesn't exist, just to see what an error status code looks like. Here's how we do it using the `requests.get()` function:

```
response = requests.get("http://api.open-notify.org/this-api-doesnt-exist")
print(response.status_code)
```

```
404
```

`doesnt-exist`, which (as expected) doesn't exist. This helps us understand what an error response looks like in practice.

Understanding Common API Status Codes

Every API request to a web server returns a status code indicating the outcome of the request. Here are some common codes relevant to GET requests:

- `200`: Everything went okay, and the result has been returned (if any).
- `301`: The server is redirecting you to a different endpoint. This can happen when a company switches domain names or changes an endpoint name.
- `400`: The server thinks you made a bad request. This happens when you send incorrect data or make other client-side errors.
- `401`: The server thinks you're not authenticated. Many APIs require login credentials, so this happens when the correct credentials are not sent.
- `403`: The resource you're trying to access is forbidden. You don't have the right permissions to see it.
- `404`: The resource you tried to access wasn't found on the server.
- `503`: The server is not ready to handle the request.

Status codes starting with `4` indicate client-side errors, while codes beginning with `5` point to server-side issues. The first digit of the status code helps you quickly identify if a request was successful (`2xx`) or if there was an error (`4xx` or `5xx`). Read more about status codes here to deepen your understanding.

Now that you understand the basics of API requests and status codes, you're ready to start making your own requests and handling the responses!

How to Read API Documentation (and Why It Matters)

Consulting Documentation for Successful API Requests

When working with APIs, especially in the context of AI and data science, reviewing the documentation is essential for making effective requests. Documentation from providers like OpenAI, Google Cloud AI, or IBM Watson outlines how to use their services efficiently.

Key Elements of API Documentation

Most API documentation includes details on available endpoints, required parameters, authentication methods, and expected response formats. For instance, the OpenAI API documentation provides comprehensive guidance on using their language models, such as GPT-4, for tasks like natural language processing.

Exploring the Open Notify API

Here, we'll work with the Open Notify API, which provides data about the International Space Station. This API is ideal for beginners due to its straightforward design and lack of authentication requirements.

Understanding API Endpoints

APIs often offer multiple endpoints for different types of data. We'll start with the http://api.open-notify.org/astros.json endpoint, which returns information about astronauts currently in space. According to the documentation, this API requires no inputs, making it simple to use.

```
response = requests.get("http://api.open-notify.org/astros")
print(response.status_code)
```

```
200
```

The `200` status code indicates that our request was successful. The API response is in JSON format. To view the data, we'll use the `response.json()` method:

```
print(response.json())
```

```
{'craft': 'ISS', 'name': 'Tracy Caldwell Dyson'},
{'craft': 'ISS', 'name': 'Matthew Dominick'},
{'craft': 'ISS', 'name': 'Michael Barratt'},
{'craft': 'ISS', 'name': 'Jeanette Epps'},
{'craft': 'ISS', 'name': 'Alexander Grebenkin'},
{'craft': 'ISS', 'name': 'Butch Wilmore'},
{'craft': 'ISS', 'name': 'Sunita Williams'},
{'craft': 'Tiangong', 'name': 'Li Guangsu'},
{'craft': 'Tiangong', 'name': 'Li Cong'},
{'craft': 'Tiangong', 'name': 'Ye Guangfu'}],
'number': 12, 'message': 'success'}
```


The Importance of API Documentation

Thoroughly understanding API documentation is key to leveraging the capabilities of APIs for AI and data science. By carefully reviewing the guidelines, you can ensure you're fully prepared to make the most of these tools.

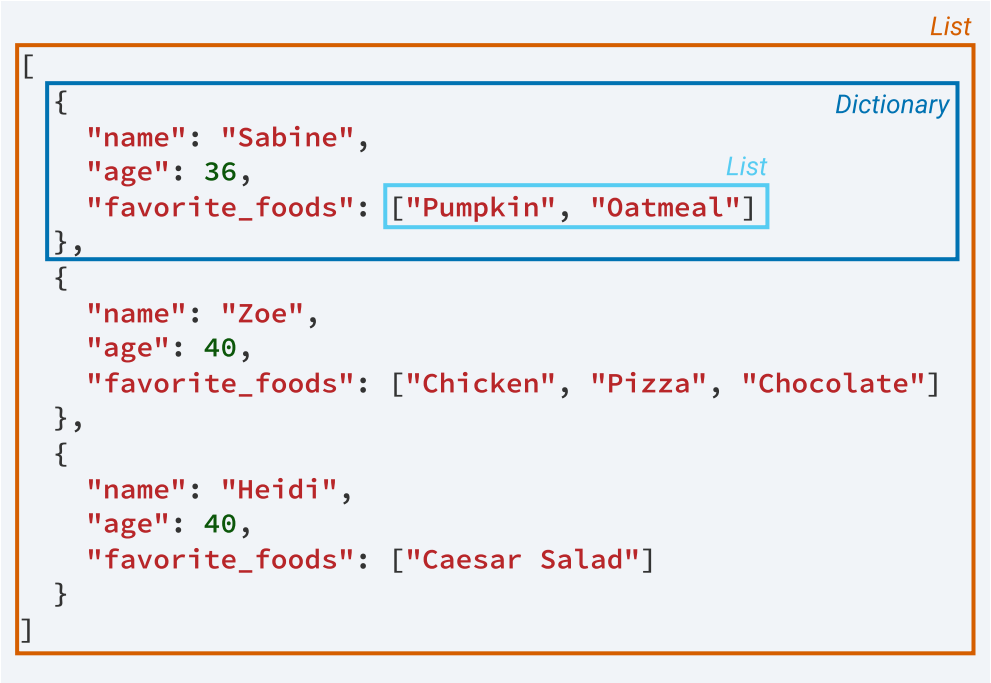
How to Handle JSON from an API in Python

What is JSON?

JSON (**JavaScript Object Notation**) is widely used in APIs to encode data structures for easy machine readability. APIs primarily exchange data in JSON format, making it a cornerstone for modern data-driven applications.

such as those from OpenAI, Google Cloud AI, and IBM Watson, use JSON as the standard format for requests and responses. This makes it straightforward to integrate API capabilities into Python applications.

The JSON output we received earlier from the API closely resembles Python dictionaries, lists, strings, and integers. JSON essentially represents these familiar objects as strings:



Working with JSON in Python

Python's built-in `json` package offers robust JSON support. This library allows you to convert between JSON strings and Python objects, such as lists and dictionaries, effortlessly. For example, the data about astronauts we retrieved earlier is a JSON string that can be converted into a Python dictionary.

- `json.loads()`: Converts a JSON string into a Python object.

Using `dumps()`, we can format JSON data for better readability:

```
import json

# create a formatted string of the Python JSON object
def jprint(obj):
    text = json.dumps(obj, sort_keys=True, indent=4)
    print(text)

jprint(response.json())
```

```
{
    "message": "success",
    "number": 12,
    "people": [
        {
            "craft": "ISS",
            "name": "Oleg Kononenko"
        },
        {
            "craft": "ISS",
            "name": "Nikolai Chub"
        },
        {
            "craft": "ISS",
            "name": "Tracy Caldwell Dyson"
        },
        {
            "craft": "ISS",
            "name": "Matthew Dominick"
        },
        {
            "craft": "ISS",
            "name": "Michael Barratt"
        },
        {
            "craft": "ISS",
            "name": "Jeanette Epps"
        },
        {
```

```
        {
            "craft": "ISS",
            "name": "Butch Wilmore"
        },
        {
            "craft": "ISS",
            "name": "Sunita Williams"
        },
        {
            "craft": "Tiangong",
            "name": "Li Guangsu"
        },
        {
            "craft": "Tiangong",
            "name": "Li Cong"
        },
        {
            "craft": "Tiangong",
            "name": "Ye Guangfu"
        }
    ]
}
```


The formatted output reveals that 12 people are currently in space, with their names stored in dictionaries inside a list. This makes the data easy to work with in Python.

Comparing this output with the documentation for the endpoint confirms that it matches the expected structure.

between diverse systems and allows developers to maximize the potential of multiple APIs.

Using an API with Query Parameters

The `http://api.open-notify.org/astros.json` endpoint we used earlier does not require parameters. We simply send a GET request, and the API returns data about the number of people currently in space.

However, many APIs require parameters to customize their behavior. This is especially common in APIs for AI and data science, where query parameters allow us to access specific subsets of data, configure AI models, or refine the results we receive.

How Query Parameters Work

Query parameters are a way to filter and customize the data returned by an API. For example, you might use a `filter_by` parameter to retrieve data for a specific region or topic. This approach ensures you only receive the information you need, avoiding unnecessary data transfer.

To make this concept more relatable, imagine you're ordering a burger at a restaurant. You might ask for extra cheese, no pickles, and sweet potato fries instead of regular fries. These specific instructions are like query parameters—they customize your order to fit your preferences.

Exploring the World Bank's Development Indicators API

For this example, we'll use the World Bank's Development Indicators, a database with global development data for over 200 countries. To streamline access, we've built a dedicated server (`https://api-server.dataquest.io/economic_data`) that simplifies querying this resource in our examples.

Using Query Parameters to Filter Data

Optional query parameters let us extract a subset of data from an API instead of retrieving everything. For instance, to filter data for countries in Sub-Saharan Africa, we could use the following URL:

```
https://api-server.dataquest.io/economic_data/countries?filter_by=region=Sub-Saharan%2
```

/series_time, and more.

Making Parameterized API Requests in Python

Here's an example of making a GET request without query parameters:

```
import requests

response = requests.get("https://api-server.dataquest.io/economic_data/countries")
data = response.json()
```

This retrieves a complete list of all countries in the database. To refine the results, we can add query parameters:

```
response = requests.get("https://api-server.dataquest.io/economic_data/countries?filte
data = response.json()
```

Here's a breakdown of the query string:

- `?`: Indicates the start of the query string, separating it from the base URL.
- `filter_by`: Specifies the type of filtering to apply.
- `region`: Defines the field in the database to filter (in this case, the geographical region).
- `=`: Assigns the value to the specified field (`Sub-Saharan Africa`).
- `%20`: Represents a space in the URL, as spaces cannot be included directly.

Wrapping Up: Using an API in Python

APIs provide an efficient way to access real-time data and integrate it into Python projects, from tracking the ISS's location to building machine learning models. By making API requests and working with JSON data, you can add dynamic possibilities to your data science and AI applications.

For example, real-time ISS pass time data could be used to train a machine learning model that predicts future pass times for any location. Building such a project might involve setting up a data pipeline to regularly fetch API data, preprocess it, and feed it into the model.

Knowing how to interact with APIs in Python and being able to parse their responses are fundamental skills for working in data science and AI. It enables the seamless integration of multiple data sources, significantly expanding the scope of what you can achieve with Python.

Next Steps to Expand Your Python API Skills

Ready to deepen your knowledge? Explore our interactive APIs and Webscraping course, perfect for honing your API skills. If you're just starting, our Introduction to Python Programming course is a great place to begin. For those focused on AI, check out our Generative AI Fundamentals in Python skill path to explore cutting-edge techniques.

API Tutorials

About the author

Charlie Custer

Charlie is a student of data science, and also a content marketer at Dataquest. In his free time, he's learning to mountain bike and making videos about it.

More learning resources

Tutorial: Python Regex (Regular Expressions) for Data Scientists

Read more

Python Tutorial: Web Scraping with Scrapy (8 Code Examples)

About

Career Paths

Business Analyst Tableau

Junior Data Analyst

Skill Paths

SQL Courses

AI Courses

Machine Learning Courses

Deep Learning Courses

Excel Courses

Statistics Courses

Explore

Course Catalog

Projects

Teaching Method

Project-first Learning

How to Learn Python

The Dataquest Download